

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Sementara
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Modul 1
Pengantar Mikrokontroler

Lu'bah Al 'Aini - 5024241082

2026

1 Pendahuluan

1.1 Latar Belakang

Sistem tertanam (*embedded system*) kini hadir di hampir semua perangkat yang kita gunakan sehari-hari, mulai dari peralatan rumah tangga, kendaraan, perangkat medis, hingga perangkat *Internet of Things* (IoT) dan robot. Inti dari sistem-sistem tersebut adalah mikrokontroler, yaitu komputer kecil dalam satu chip yang memiliki prosesor, memori, dan periferal masukan/keluaran sendiri. Karena itu, kemampuan memilih, memprogram, dan menghubungkan mikrokontroler dengan dunia luar menjadi kompetensi dasar bagi mahasiswa Teknik Komputer.

Di dunia industri, tidak ada satu jenis mikrokontroler yang dipakai untuk semua kebutuhan. Keluarga STM32 berbasis ARM Cortex-M banyak dipakai pada sistem kendali dan perangkat industri, sedangkan ESP32 populer untuk perangkat IoT karena sudah dilengkapi WiFi dan Bluetooth. Kedua keluarga ini juga dapat diprogram dengan beberapa *framework*, misalnya Arduino yang sederhana dan ESP-IDF yang lebih dekat ke perangkat keras. Seorang pengembang perlu memahami perbedaan cara *flashing*, struktur program, dan tingkat abstraksi dari masing-masing pilihan tersebut agar dapat menentukan alat yang tepat untuk sebuah proyek.

Selain itu, interaksi mikrokontroler dengan dunia nyata hampir selalu melalui GPIO (*General Purpose Input/Output*). Permasalahan yang sering muncul pada tahap ini cukup mendasar namun berdampak besar, seperti pin masukan yang mengambang (*floating*) sehingga terbaca acak, tombol mekanik yang terbaca berkali-kali akibat *bouncing*, serta kebutuhan membaca besaran yang bersifat kontinu seperti intensitas cahaya. Praktikum Modul 1 ini dilaksanakan untuk membangun fondasi tersebut: menyiapkan lingkungan pengembangan PlatformIO, melakukan *flashing* ke STM32 Blackpill dan ESP32, serta menggunakan GPIO dalam tiga mode akses, yaitu keluaran digital, masukan digital (dengan resistor *pull-up/pull-down* dan *debouncing*), dan masukan analog melalui ADC.

1.2 Dasar Teori

1.2.1 Mikrokontroler dan GPIO

Mikrokontroler adalah rangkaian terintegrasi yang memuat CPU, memori program (Flash), memori data (RAM), serta berbagai periferal seperti GPIO, timer, ADC, dan antarmuka komunikasi dalam satu chip. Berbeda dengan mikroprosesor pada komputer umum, mikrokontroler dirancang untuk menjalankan satu tugas khusus secara terus-menerus dengan konsumsi daya rendah. GPIO merupakan pin serbaguna yang fungsinya ditentukan oleh perangkat lunak. Pada praktikum ini GPIO digunakan dalam tiga mode, yaitu:

1. **Keluaran digital**, mikrokontroler menetapkan level tegangan pin menjadi HIGH atau LOW, misalnya untuk menyalakan LED.
2. **Masukan digital**, mikrokontroler membaca level tegangan pin dan menerjemahkannya menjadi logika 1 atau 0, misalnya untuk membaca status tombol.
3. **Masukan analog**, mikrokontroler mengukur besar tegangan pada pin melalui ADC sehingga diperoleh nilai bertingkat, bukan hanya dua kondisi.

1.2.2 PlatformIO dan Visual Studio Code

PlatformIO adalah ekosistem pengembangan sistem tertanam lintas platform yang dapat dipasang sebagai ekstensi pada Visual Studio Code. PlatformIO mengelola *toolchain* (kompiler, *uploader*, dan *debugger*), paket *platform*, *framework*, dan pustaka secara otomatis. Seluruh konfigurasi sebuah proyek disimpan dalam berkas `platformio.ini`, yang berisi tiga parameter utama: `platform` (keluarga chip, misalnya `ststm32` atau `espressif32`), `board` (jenis papan), dan `framework` (misalnya `arduino` atau `espidf`). Dengan mengubah isi berkas ini, satu proyek yang sama dapat dipakai untuk chip atau *framework* yang berbeda. Struktur proyek PlatformIO terdiri atas folder `src/` untuk kode utama, `include/` untuk berkas *header*, `lib/` untuk pustaka lokal, dan `platformio.ini` sebagai berkas konfigurasi.

1.2.3 STM32 Blackpill, ST-Link, dan Framework Arduino

STM32 Blackpill adalah papan pengembangan berbasis STM32F401CCU6 dengan inti ARM Cortex-M4 32-bit yang dilengkapi FPU, berkecepatan hingga 84 MHz, memori Flash 256 KB, dan RAM 64 KB. LED bawaan papan terhubung ke pin PC13 dengan logika aktif LOW, artinya LED menyala ketika pin diberi logika LOW.

Blackpill tidak dilengkapi programmer/debugger di atas papan, sehingga proses *flashing* umumnya dilakukan menggunakan ST-Link melalui protokol SWD (*Serial Wire Debug*). SWD hanya membutuhkan dua jalur sinyal, yaitu SWDIO (data dua arah, pin PA13) dan SWCLK (*clock*, pin PA14), ditambah GND sebagai referensi dan 3,3 V. Melalui SWD, ST-Link dapat menulis Flash secara langsung sekaligus melakukan *debugging* seperti menghentikan program dan membaca isi register.

Pada praktikum ini STM32 diprogram menggunakan *framework* Arduino versi STM32 (STM32duino). *Framework* ini menyediakan fungsi yang sama seperti Arduino pada umumnya, yaitu `pinMode()`, `digitalWrite()`, dan `delay()`, dan di baliknya tetap memanggil pustaka HAL (*Hardware Abstraction Layer*) dari STMicroelectronics. STM32duino juga mendukung penamaan pin sesuai label port, misalnya `PC13`.

1.2.4 ESP32 dan Framework ESP-IDF

ESP32 adalah mikrokontroler 32-bit dari Espressif dengan prosesor *dual-core* Xtensa LX6 serta WiFi dan Bluetooth terintegrasi. Papan ESP32 DevKit v1 dilengkapi chip USB-to-Serial (umumnya CP2102 atau CH340) dan rangkaian *auto-reset*, sehingga dapat diprogram langsung melalui kabel USB tanpa programmer tambahan.

ESP-IDF (*Espressif IoT Development Framework*) adalah *framework* resmi Espressif yang dibangun di atas sistem operasi waktu nyata FreeRTOS. Program ESP-IDF tidak menggunakan pasangan `setup()` dan `loop()`, melainkan dimulai dari fungsi `app_main()` yang berjalan sebagai sebuah *task* FreeRTOS. Penundaan waktu dilakukan dengan `vTaskDelay()`, yang menyerahkan CPU ke *task* lain selama menunggu. Akses GPIO dilakukan melalui *driver* seperti `gpio_set_direction()` dan `gpio_set_level()`. Perbandingan singkat kedua *framework* ditunjukkan pada Tabel 1.

Tabel 1: Perbandingan *framework* Arduino dan ESP-IDF

Aspek	**Arduino**	**ESP-IDF**
Titik masuk program	`setup()` dan `loop()`	`app_main()`
Penundaan waktu	`delay(ms)`	`vTaskDelay( pdMS_TO_TICKS(ms))`
Tingkat abstraksi	Tinggi, mudah dipelajari	Rendah–menengah, lebih dekat ke perangkat keras
Akses FreeRTOS	Tersembunyi atau terbatas	Langsung (*task*, *queue*, *semaphore*)

1.2.5 Resistor Pull-up dan Pull-down

Pin masukan digital memiliki impedansi yang sangat tinggi. Jika pin tidak terhubung ke level tegangan apa pun, misalnya saat tombol tidak ditekan, pin berada dalam kondisi mengambang (*floating*) dan nilainya dapat berubah-ubah secara acak akibat derau atau medan listrik di sekitarnya. Untuk mencegah hal ini, pin ditarik ke level tegangan tertentu menggunakan resistor:

- **Pull-up**: pin dihubungkan ke VCC (3,3 V) melalui resistor dan tombol dihubungkan ke GND. Pin terbaca HIGH saat tombol tidak ditekan dan LOW saat ditekan (*active-low*).
- **Pull-down**: pin dihubungkan ke GND melalui resistor dan tombol dihubungkan ke VCC. Pin terbaca LOW saat tombol tidak ditekan dan HIGH saat ditekan (*active-high*).

Resistor tersebut dapat dipasang secara eksternal di *breadboard* (umumnya 10 kΩ) atau diaktifkan secara internal di dalam chip. Pada ESP32 dengan *framework* Arduino, resistor internal diaktifkan dengan `pinMode(pin, INPUT_PULLUP)` atau `pinMode(pin, INPUT_PULLDOWN)`. Pin GPIO34–39 pada ESP32 bersifat *input-only* dan tidak memiliki resistor *pull* internal, sehingga wajib memakai resistor eksternal.

1.2.6 Bouncing dan Debouncing

Tombol mekanik terdiri atas dua kontak logam yang saling bersentuhan saat ditekan. Ketika bersentuhan atau terlepas, kontak tersebut memantul beberapa kali sebelum benar-benar stabil, sehingga sinyal berganti HIGH–LOW berulang kali dalam rentang waktu beberapa milidetik. Fenomena ini disebut *bouncing*. Karena mikrokontroler membaca pin jutaan kali per detik, satu kali tekan dapat terhitung sebagai beberapa kali tekan. *Debouncing* adalah teknik untuk memastikan hanya satu perubahan yang sah dicatat, misalnya dengan menunggu sinyal tetap stabil selama selang waktu tertentu (umumnya 20–50 ms) sebelum perubahan diterima. *Debouncing* dapat dilakukan secara perangkat lunak maupun perangkat keras.

1.2.7 ADC dan Pembagi Tegangan LDR

ADC (*Analog-to-Digital Converter*) mengubah tegangan analog yang bersifat kontinu menjadi bilangan digital diskret. ADC dengan resolusi n bit menghasilkan 2^n tingkat nilai. Hubungan antara nilai digital (*raw*) dan tegangan masukan dinyatakan dengan:

$$V_{in} \approx \text{raw} \times \frac{V_{ref}}{2^n - 1} \tag{1}$$

ESP32 memiliki ADC SAR dengan resolusi bawaan 12 bit (nilai 0–4095) dan rentang kerja sekitar 0–3,3 V, yang dibaca dengan fungsi `analogRead()`. Pin ADC1 (misalnya GPIO32–39) lebih disarankan karena ADC2 tidak dapat digunakan bersamaan dengan WiFi.

LDR (*Light Dependent Resistor*) adalah resistor yang nilai hambatannya turun ketika intensitas cahaya meningkat. Agar perubahan hambatan dapat dibaca ADC, LDR dirangkai sebagai pembagi tegangan bersama resistor $R_1 = 10$ kΩ. Pada rangkaian praktikum, LDR terhubung ke 3,3 V dan R_1 ke GND, sehingga tegangan titik tengah adalah:

$$V_o = V_{in} \times \frac{R_1}{R_{LDR} + R_1} \tag{2}$$

Saat terang, R_{LDR} mengecil sehingga V_o dan nilai ADC naik. Saat gelap, R_{LDR} membesar sehingga V_o dan nilai ADC turun.

2 Tugas Pendahuluan

1. Jelaskan perbedaan mendasar antara framework Arduino dan ESP-IDF dalam memprogram mikrokontroler, termasuk kelebihan dan kekurangan masing-masing.

Jawaban:

Perbedaan mendasarnya terletak pada tingkat abstraksi dan cara program dijalankan. *Framework* Arduino adalah lapisan abstraksi tingkat tinggi yang menyembunyikan detail perangkat keras di balik fungsi sederhana seperti `pinMode()`, `digitalWrite()`, dan `delay()`. Program ditulis dalam dua fungsi, `setup()` yang dijalankan sekali dan `loop()` yang diulang terus-menerus. Pada ESP32, *framework* Arduino sebenarnya dibangun di atas ESP-IDF, dan fungsi `loop()` dijalankan di dalam sebuah *task* FreeRTOS yang tidak terlihat oleh pengguna.

ESP-IDF adalah *framework* resmi dan *native* dari Espressif. Program dimulai dari `app_main()` dan pengguna berinteraksi langsung dengan FreeRTOS (membuat *task*, *queue*, *semaphore*) serta *driver* periferal yang lebih rinci. Konfigurasi sistem juga dapat diatur melalui `menuconfig`.

Tabel 2: Kelebihan dan kekurangan *framework* Arduino dan ESP-IDF

	Arduino	**ESP-IDF**
Kelebihan	Mudah dipelajari dan cepat untuk membuat prototipe; kode portabel antarpapan (AVR, STM32, ESP32); komunitas dan pustaka sangat banyak.	Akses penuh ke seluruh fitur ESP32 (WiFi/BLE tingkat rendah, *power management*, *dual-core*); FreeRTOS dapat dipakai langsung sehingga cocok untuk *multitasking*; dukungan resmi dan pembaruan dari Espressif; lebih cocok untuk produk akhir.
Kekurangan	Kontrol terhadap perangkat keras terbatas; `delay()` bersifat *blocking* sehingga kurang cocok untuk program kompleks; ukuran biner dan *overhead* lebih besar; versi IDF di bawahnya sering tertinggal.	Kurva belajar lebih curam; kode lebih panjang untuk tugas sederhana; waktu *build* pertama lebih lama; kode hanya berlaku untuk chip Espressif.

Secara singkat, Arduino cocok untuk prototipe cepat dan pembelajaran awal, sedangkan ESP-IDF cocok ketika dibutuhkan kontrol penuh, performa, dan keandalan untuk aplikasi IoT yang kompleks.

2. **Mengapa board STM32 Blackpill memerlukan programmer eksternal (ST-Link) untuk diflash, sedangkan ESP32 dapat diprogram langsung melalui kabel USB? Kaitkan jawaban Anda dengan keberadaan chip USB-to-Serial.**

   Jawaban:

   Komputer berkomunikasi melalui USB, sedangkan mikrokontroler menerima program melalui antarmuka yang berbeda. Oleh karena itu dibutuhkan perangkat penerjemah di antara keduanya.

   Pada papan **ESP32 DevKit**, penerjemah tersebut sudah tersedia di atas papan, yaitu chip USB-to-Serial (CP2102 atau CH340). Chip ini mengubah data USB dari komputer menjadi sinyal UART (TX/RX) yang dapat dibaca ESP32. ESP32 sendiri memiliki *bootloader* di ROM yang mampu menerima program baru melalui UART. Chip USB-to-Serial juga mengendalikan pin EN dan GPIO0 melalui sinyal DTR/RTS (rangkaian *auto-reset*), sehingga *tool* `esptool` dapat otomatis memasukkan ESP32 ke mode *download* lalu me-*reset*-nya setelah *flashing*. Karena itu cukup satu kabel USB untuk memprogram ESP32.

   Pada **STM32 Blackpill**, tidak terdapat chip USB-to-Serial maupun *debugger* di atas papan. Port USB-C pada Blackpill terhubung langsung ke periferal USB milik STM32 (PA11/PA12), bukan ke chip penerjemah. Port ini hanya dapat dipakai untuk *upload* jika program yang berjalan sudah mengimplementasikan USB, atau jika chip dimasukkan secara manual ke mode DFU dengan menahan tombol BOOT0, yang kurang praktis dan tidak menyediakan fitur *debugging*. Cara standar untuk memprogram STM32 adalah melalui antarmuka SWD (SWDIO dan SWCLK). ST-Link berfungsi sebagai jembatan USB-ke-SWD yang dapat menulis langsung ke memori Flash tanpa bergantung pada program yang sedang berjalan, sekaligus menyediakan fitur *debugging* seperti *breakpoint* dan pembacaan register.

   Kesimpulannya, ESP32 DevKit dapat diprogram langsung karena sudah memiliki chip USB-to-Serial dan rangkaian *auto-reset*, sedangkan Blackpill tidak memiliki perangkat penerjemah tersebut sehingga memerlukan ST-Link sebagai programmer eksternal melalui SWD.

3. **Jelaskan perbedaan kondisi logika (HIGH/LOW) pada konfigurasi pull-up dan pull-down resistor, baik saat tombol tidak ditekan maupun saat ditekan.**

   Jawaban:

   - **Pull-up**: resistor menghubungkan pin GPIO ke 3,3 V, sedangkan tombol menghubungkan pin ke GND. Saat tombol **tidak ditekan**, tidak ada arus yang mengalir melalui resistor sehingga tidak ada tegangan jatuh, dan pin bernilai sama dengan VCC, yaitu **HIGH**. Saat tombol **ditekan**, pin terhubung langsung ke GND sehingga terbaca **LOW**. Resistor membatasi arus dari VCC ke GND agar tidak terjadi hubung singkat. Konfigurasi ini disebut *active-low*.
   - **Pull-down**: resistor menghubungkan pin GPIO ke GND, sedangkan tombol menghubungkan pin ke 3,3 V. Saat tombol **tidak ditekan**, pin ditarik ke GND sehingga terbaca **LOW**. Saat tombol **ditekan**, pin terhubung ke 3,3 V sehingga terbaca **HIGH**. Konfigurasi ini disebut *active-high*.

Tabel 3: Kondisi logika pin pada konfigurasi *pull-up* dan *pull-down*

Konfigurasi	Tombol tidak ditekan	Tombol ditekan
Pull-up (resistor ke VCC, tombol ke GND)	HIGH	LOW
Pull-down (resistor ke GND, tombol ke VCC)	LOW	HIGH

Logika kedua konfigurasi saling berkebalikan, sehingga kode program harus menyesuaikan. Pada *pull-up*, kondisi tombol ditekan dideteksi dengan `digitalRead(pin) == LOW`, sedangkan pada *pull-down* dengan `digitalRead(pin) == HIGH`. Hasil ini berlaku sama untuk resistor eksternal maupun internal.

4. **Jelaskan fenomena bouncing pada tombol mekanik, mengapa hal ini bisa terjadi, dan sebutkan minimal dua pendekatan untuk mengatasinya.**

   Jawaban:

   Bouncing adalah kondisi ketika satu kali penekanan atau pelepasan tombol menghasilkan beberapa transisi sinyal HIGH–LOW yang sangat cepat sebelum sinyal stabil. Hal ini terjadi karena kontak di dalam tombol terbuat dari logam yang memiliki elastisitas. Saat kedua kontak bertemu, keduanya saling memantul seperti bola yang dijatuhkan ke lantai, sehingga sambungan listrik terputus dan tersambung berulang kali dalam waktu beberapa mikrodetik hingga belasan milidetik. Permukaan kontak yang tidak rata atau kotor dan cara menekan yang ragu-ragu dapat memperparah efek ini. Karena mikrokontroler membaca pin dengan sangat cepat, setiap pantulan dapat terbaca sebagai penekanan baru, sehingga satu kali tekan dapat terhitung lebih dari satu.

   Pendekatan untuk mengatasinya antara lain:

   a. **Debouncing perangkat lunak berbasis waktu**: setiap kali terdeteksi perubahan, catat waktunya dengan `millis()`. Perubahan baru diterima jika sinyal tetap stabil lebih lama dari batas tertentu (misalnya 50 ms). Cara ini *non-blocking* dan digunakan pada Percobaan 4.
   b. **Debouncing perangkat lunak berbasis sampling/penghitung**: pin dibaca secara berkala, misalnya setiap 5 ms, dan perubahan baru diterima jika beberapa pembacaan berturut-turut menunjukkan nilai yang sama.
   c. **Debouncing perangkat keras dengan filter RC dan Schmitt trigger**: kapasitor dan resistor meredam perubahan cepat sehingga sinyal berubah secara halus, lalu Schmitt trigger (misalnya IC 74HC14) mengubahnya kembali menjadi sinyal digital yang bersih berkat histeresisnya.
   d. **Latch SR**: untuk saklar dua posisi (SPDT), latch SR mengunci keadaan pada kontak pertama sehingga pantulan berikutnya diabaikan.

   Pemilihan waktu *debounce* adalah kompromi: nilai terlalu kecil masih meloloskan *bouncing*, sedangkan nilai terlalu besar membuat tombol terasa lambat dan penekanan cepat bisa tidak terdeteksi.

5. **Bagaimana ADC (Analog-to-Digital Converter) mengubah sinyal analog menjadi nilai digital? Jelaskan pula pengaruh resolusi bit ADC terhadap presisi hasil pembacaan.**

   Jawaban:

   ADC mengubah sinyal analog menjadi digital melalui tiga tahap:

   a. **Sampling (pencuplikan)**: tegangan masukan diambil pada saat tertentu dan ditahan sementara oleh rangkaian *sample-and-hold* agar tidak berubah selama proses konversi.
   b. **Kuantisasi**: tegangan yang ditahan dibandingkan dengan tegangan referensi dan dipetakan ke tingkat diskret terdekat. ESP32 menggunakan ADC jenis SAR (*Successive Approximation Register*), yang menentukan nilai bit demi bit dari MSB ke LSB. Setiap langkah membandingkan tegangan masukan dengan tegangan keluaran DAC internal, mirip pencarian biner.

c. **Pengkodean**: tingkat hasil kuantisasi dinyatakan sebagai bilangan biner yang dapat dibaca program, misalnya melalui `analogRead()`.

Resolusi n bit menentukan jumlah tingkat yang tersedia, yaitu 2^n. Selisih tegangan terkecil yang dapat dibedakan (1 LSB) adalah:

$$\text{LSB} = \frac{V_{ref}}{2^n} \tag{3}$$

Tabel 4: Pengaruh resolusi ADC terhadap presisi pada $V_{ref} = 3{,}3$ V

Resolusi	Jumlah tingkat	1 LSB
8 bit	256	$\approx 12{,}9$ mV
10 bit	1024	$\approx 3{,}22$ mV
12 bit (ESP32)	4096	$\approx 0{,}81$ mV

Semakin tinggi resolusi, semakin kecil langkah tegangan yang dapat dibedakan sehingga kesalahan kuantisasi (maksimal $\pm\frac{1}{2}$ LSB) semakin kecil dan hasil pembacaan semakin presisi. Sebagai contoh, pada ADC 12 bit ESP32, nilai *raw* 2048 setara dengan $2048 \times 3{,}3/4095 \approx 1{,}65$ V. Namun resolusi tinggi tidak otomatis berarti akurat. Derau, ketidaklinearan, dan kestabilan tegangan referensi juga memengaruhi hasil. Pada ESP32, ADC diketahui kurang linear di dekat 0 V dan di dekat batas atas rentangnya, sehingga perataan beberapa sampel atau kalibrasi tetap diperlukan untuk hasil yang lebih akurat.

6. Lakukan set-up untuk tools: vscode, ekstensi platform-io (dengan framework esp-idf dan stm32), dan ekstensi serial-plotter pada laptop kalian, berikan screenshoot bahwa laptop kalian sudah ter-set-up dengan baik.

Jawaban:

Langkah instalasi yang dilakukan:

a. Mengunduh dan memasang Visual Studio Code dari situs resminya.

b. Memasang ekstensi **PlatformIO IDE** melalui menu *Extensions* (`Ctrl+Shift+X`), lalu menunggu PlatformIO Core selesai terpasang hingga ikon PlatformIO muncul di *sidebar*.

c. Memasang platform ESP32 dan STM32 melalui terminal PlatformIO:

```
pio pkg install --platform espressif32
pio pkg install --platform ststm32
```

d. Membuat proyek uji dengan `framework = espidf` (papan `esp32dev`) dan `framework = arduino` (papan `blackpill_f401cc`), lalu memastikan keduanya berhasil di-*build*.

e. Memasang *driver* USB (CP210x/CH340 untuk ESP32 dan ST-Link untuk STM32).

f. Memasang ekstensi **Serial Plotter** (`badlogicgames.serial-plotter`) dan memverifikasinya melalui `Ctrl+Shift+P` → *Serial Plotter: Open pane*.